

Web Scraper for RAG Pipeline

1. Project Repository:

<https://github.com/GautamKumar2005/Small-backend/tree/main/RAG>

2. Language & Tools Used

Programming Language: Python (v3.12)

Libraries / Tools:

- **requests:** For handling HTTP communication and sessions.
- **urllib3:** Utilized for robust retry logic and network backoff.
- **BeautifulSoup4 (bs4):** For parsing HTML DOM and extracting text.
- **urllib.robotparser:** Native Python tool to strictly adhere to robots.txt.

3. Steps to Proceed (How to Process)

To execute the scraper locally, follow these steps in your terminal:

Step 1: Install the required dependencies.

```
pip install -r requirements.txt
```

Step 2: Run the scraping script.

```
python scraper.py
```

The script will automatically parse robots.txt, begin traversing the site using a 1-second polite rate limit, and save the data.

4. How Output Comes

The output is generated incrementally and saved into a line-delimited JSON format file named **corpus.jsonl**. This format is the industry standard for RAG embeddings.

An example of a single extracted record looks like this:

```
{"text": "The world as we have created it is a process of our  
thinking...", "author": "Albert Einstein", "tags": ["change",  
"deep-thoughts", "thinking", "world"], "source_url":  
"http://quotes.toscrape.com/", "scraped_at":
```

"2026-07-18T16:13:33.797455Z"}}

5. Conclusion

This project successfully demonstrates the construction of a professional-grade web scraper. By implementing robots.txt compliance, resilience against server errors through automated retries, polite rate limiting, and clean text extraction stripped of artifacts, the resulting **corpus.jsonl** dataset is perfectly primed for ingestion into vector databases and embedding models for a Retrieval-Augmented Generation pipeline.